

SQL och Databasdesign – VG

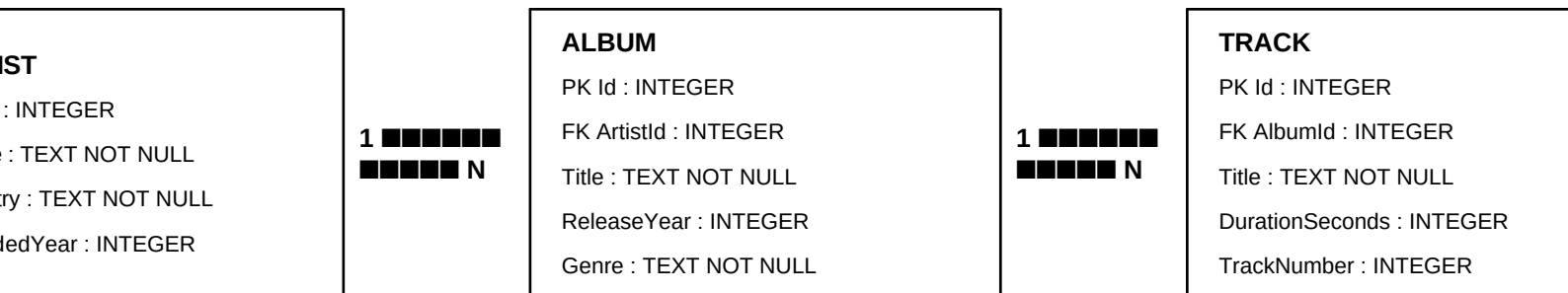
Tema: MusicLibrary

En individuell databaslösning som visar datamodellering, CREATE, INSERT, SELECT, JOIN, UPDATE, DELETE, SQL/LINQ, säkerhet och versionshantering.

1. Val av scenario

Jag har valt temat **Musikbibliotek**. Databasen består av Artist, Album och Track. Valet passar bra eftersom det naturligt innehåller både 1-n-relationer och flera nivåer av JOINS: en artist kan ha flera album och ett album kan ha flera tracks.

2. ER-diagram



Relationerna är 1-n: en Artist kan ha många Album, medan varje Album tillhör exakt en Artist. På samma sätt kan ett Album innehålla många Tracks, medan varje Track tillhör ett Album. Det behövs därför ingen separat kopplingstabell för dessa relationer.

3. Tabeller, datatyper och constraints

Tabell	Syfte	Viktiga constraints
Artist	Lagrar artister.	PK Id, UNIQUE Name, NOT NULL, CHECK FoundedYear
Album	Lagrar album och kopplar dem till artist.	PK Id, FK ArtistId, NOT NULL, CHECK ReleaseYear
Track	Lagrar låtar och kopplar dem till album.	PK Id, FK AlbumId, NOT NULL, CHECK DurationSeconds, UNIQUE (AlbumId, TrackNumber)

INTEGER används för identifierare, årtal, tracknummer och speltid eftersom dessa värden är numeriska. TEXT används för namn, titlar, länder och genre. NOT NULL används där informationen krävs. PRIMARY KEY identifierar varje rad unikt och FOREIGN KEY säkerställer att relationerna pekar på befintliga rader. CHECK begränsar uppenbart orimliga värden och UNIQUE på (AlbumId, TrackNumber) gör att samma tracknummer inte kan förekomma två gånger på samma album. ON DELETE CASCADE används mellan Album och Track eftersom tracks inte ska kunna ligga kvar utan sitt album, medan ON DELETE RESTRICT på Artist skyddar mot att en artist raderas när album fortfarande refererar till artisten.

4. CREATE – skapa tabeller

```
-- create_tables.sql
-- Database: MusicLibrary
-- Dialect: SQLite

PRAGMA foreign_keys = ON;

CREATE TABLE Artist (
  Id INTEGER PRIMARY KEY,
  Name TEXT NOT NULL UNIQUE,
  Country TEXT NOT NULL,
  FoundedYear INTEGER CHECK (FoundedYear BETWEEN 1900 AND 2100)
);

CREATE TABLE Album (
  Id INTEGER PRIMARY KEY,
  ArtistId INTEGER NOT NULL,
  Title TEXT NOT NULL,
  ReleaseYear INTEGER NOT NULL CHECK (ReleaseYear BETWEEN 1900 AND 2100),
  Genre TEXT NOT NULL,
  FOREIGN KEY (ArtistId) REFERENCES Artist(Id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
);

CREATE TABLE Track (
  Id INTEGER PRIMARY KEY,
  AlbumId INTEGER NOT NULL,
  Title TEXT NOT NULL,
  DurationSeconds INTEGER NOT NULL CHECK (DurationSeconds > 0),
  TrackNumber INTEGER NOT NULL CHECK (TrackNumber > 0),
  FOREIGN KEY (AlbumId) REFERENCES Album(Id)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
  UNIQUE (AlbumId, TrackNumber)
);
```

5. INSERT – fylla databasen med data

```
-- insert_data.sql
-- Sample data for MusicLibrary

INSERT INTO Artist (Id, Name, Country, FoundedYear) VALUES
  (1, 'The Weeknd', 'Canada', 2010),
  (2, 'Dua Lipa', 'United Kingdom', 2015),
  (3, 'Coldplay', 'United Kingdom', 1996),
  (4, 'Adele', 'United Kingdom', 2006);

INSERT INTO Album (Id, ArtistId, Title, ReleaseYear, Genre) VALUES
  (1, 1, 'After Hours', 2020, 'Pop'),
  (2, 1, 'Starboy', 2016, 'R&B'),
  (3, 2, 'Future Nostalgia', 2020, 'Pop'),
  (4, 3, 'Music of the Spheres', 2021, 'Alternative'),
  (5, 4, '30', 2021, 'Pop');

INSERT INTO Track (Id, AlbumId, Title, DurationSeconds, TrackNumber) VALUES
  (1, 1, 'Blinding Lights', 200, 1),
  (2, 1, 'Save Your Tears', 215, 2),
  (3, 1, 'In Your Eyes', 237, 3),
  (4, 2, 'Starboy', 230, 1),
  (5, 2, 'I Feel It Coming', 269, 2),
  (6, 3, 'Levitating', 203, 1),
  (7, 3, 'Physical', 193, 2),
  (8, 4, 'Higher Power', 178, 1),
  (9, 4, 'My Universe', 228, 2),
  (10, 5, 'Easy On Me', 224, 1),
  (11, 5, 'Oh My God', 225, 2),
  (12, 5, 'Can I Get It', 192, 3);
```

6. READ – SELECT

Nedan visas sex queries. De täcker WHERE, ORDER BY, LIKE och GROUP BY. GROUP BY används endast för gruppberäkning, i detta fall COUNT per genre.

```
-- select_basic.sql
-- 6 basic SELECT queries demonstrating WHERE, ORDER BY, LIKE and GROUP BY.

-- 1. WHERE: Find tracks longer than 220 seconds.
SELECT Id, Title, DurationSeconds
FROM Track
WHERE DurationSeconds > 220;

-- 2. ORDER BY: Show albums from newest to oldest.
SELECT Id, Title, ReleaseYear, Genre
FROM Album
ORDER BY ReleaseYear DESC;

-- 3. LIKE: Find artists whose name contains the word "The".
SELECT Id, Name, Country
FROM Artist
WHERE Name LIKE '%The%';

-- 4. WHERE + ORDER BY: Find pop albums released from 2020 onwards.
SELECT Id, Title, ReleaseYear
FROM Album
WHERE Genre = 'Pop' AND ReleaseYear >= 2020
ORDER BY ReleaseYear DESC, Title ASC;

-- 5. LIKE: Find tracks containing the word "My".
SELECT Id, Title
FROM Track
WHERE Title LIKE '%My%';

-- 6. GROUP BY + COUNT: Count how many albums each genre has.
SELECT Genre, COUNT(*) AS AlbumCount
FROM Album
GROUP BY Genre;
```

7. JOIN – kombinera tabeller

Den första JOIN-frågan går från Track till Album och vidare till Artist. Den andra räknar album per artist. Det visar att foreign keys kan användas för att hämta sammanhängande information från flera tabeller.

```
-- select_join.sql
-- JOIN queries combining related tables.

-- 1. Show every track together with its album and artist.
SELECT
    Track.Title AS TrackTitle,
    Album.Title AS AlbumTitle,
    Artist.Name AS ArtistName
FROM Track
JOIN Album ON Track.AlbumId = Album.Id
JOIN Artist ON Album.ArtistId = Artist.Id
ORDER BY Artist.Name, Album.Title, Track.TrackNumber;

-- 2. Show each artist and the number of albums they have.
SELECT
    Artist.Name AS ArtistName,
    COUNT(Album.Id) AS AlbumCount
FROM Artist
JOIN Album ON Artist.Id = Album.ArtistId
GROUP BY Artist.Id, Artist.Name
ORDER BY AlbumCount DESC, Artist.Name ASC;
```

8. UPDATE och DELETE

UPDATE ändrar befintliga rader och DELETE tar bort en specifik rad med ett tydligt WHERE-villkor.

```
-- updates.sql
-- Two UPDATE operations.

-- 1. Increase the duration of one track by 5 seconds.
UPDATE Track
SET DurationSeconds = DurationSeconds + 5
WHERE Id = 1;

-- 2. Correct the genre of one album.
UPDATE Album
SET Genre = 'Pop/R&B'
WHERE Id = 2;

-- deletes.sql
-- One DELETE operation.
-- Track 12 is removed. Because Track has no dependent tables,
-- the deletion does not break any other relationship.

DELETE FROM Track
WHERE Id = 12;
```

9. SQL och LINQ

LINQ-JÄMFÖRELSE

1. SQL: WHERE + ORDER BY

SQL:
SELECT Id, Title, DurationSeconds
FROM Track
WHERE DurationSeconds > 220
ORDER BY DurationSeconds DESC;

LINQ:
var result = context.Tracks
 .Where(t => t.DurationSeconds > 220)
 .OrderByDescending(t => t.DurationSeconds)
 .Select(t => new
 {
 t.Id,
 t.Title,
 t.DurationSeconds
 })
 .ToList();

Mappning:
- WHERE -> .Where(...)
- ORDER BY ... DESC -> .OrderByDescending(...)
- SELECT kolumner -> .Select(...)
- Kör frågan och materialiserar resultatet -> .ToList()

2. SQL: LIKE

SQL:
SELECT Id, Title
FROM Track
WHERE Title LIKE '%My%';

LINQ:
var result = context.Tracks
 .Where(t => EF.Functions.Like(t.Title, "%My%"))
 .Select(t => new
 {
 t.Id,
 t.Title
 })
 .ToList();

Mappning:
- LIKE -> EF.Functions.Like(...)
- WHERE-villkoret -> .Where(...)
- Valda kolumner -> .Select(...)

3. SQL: JOIN

SQL:
SELECT
 Track.Title AS TrackTitle,
 Album.Title AS AlbumTitle,
 Artist.Name AS ArtistName
FROM Track
JOIN Album ON Track.AlbumId = Album.Id
JOIN Artist ON Album.ArtistId = Artist.Id
ORDER BY Artist.Name, Album.Title;

LINQ:
var result = context.Tracks
 .Join(
 context.Albums,
 track => track.AlbumId,
 album => album.Id,
 (track, album) => new { track, album }
)
 .Join(
 context.Artists,
 x => x.album.ArtistId,
 artist => artist.Id,
 (x, artist) => new
 {
 TrackTitle = x.track.Title,

```
        AlbumTitle = x.album.Title,  
        ArtistName = artist.Name  
    }  
)  
.OrderBy(x => x.ArtistName)  
.ThenBy(x => x.AlbumTitle)  
.ToList();
```

Mappning:

- JOIN ... ON -> .Join(... key selector ...)
- SELECT med alias -> anonymt objekt i .Select/projektionsdelen
- ORDER BY -> .OrderBy(...)
- Andra sorteringsnivån -> .ThenBy(...)

10. Säkerhet

Säkerhet

Säker åtkomst till databasen är viktig eftersom en databas ofta innehåller information som inte ska kunna läsas eller ändras av obehöriga. Authentication betyder att systemet verifierar vem en användare är, medan authorization avgör vad den användaren får göra efter inloggningen. I ett backendprojekt bör databasanvändare ha så få rättigheter som möjligt enligt principen om least privilege. SQL-frågor ska normalt skickas med parametrar eller via ORM-frågor så att SQL injection försvåras. Känsliga uppgifter ska inte lagras i klartext och hemligheter som lösenord, API-nycklar och connection strings ska inte hårdkodas i källkoden. Åtkomst till databasen bör dessutom skyddas med säkra anslutningar, korrekt secrets-hantering och regelbundna säkerhetskopior.

11. Versionshantering

Alla SQL-filer ligger i mappen `/sql`, vilket gör projektet lätt att navigera och versionshantera. Meningsfulla commits beskriver vad som faktiskt ändrats, till exempel skapandet av tabeller eller tillägg av JOIN-frågor. I ett större projekt är det bra att arbeta med branches för större förändringar och sedan merge:a färdiga ändringar till main efter granskning. Git gör det möjligt att se historik, återställa tidigare versioner och samarbeta utan att förlora kontroll över databasens utveckling. För databasutveckling är detta extra viktigt eftersom förändringar i tabellstruktur och data kan påverka hela backend-systemet.

12. Reflektion

Reflektion

Det som gick bra var att bygga databasen kring tydliga relationer mellan Artist, Album och Track. Primärnycklar och främmande nycklar gör att data hänger ihop på ett kontrollerat sätt, samtidigt som constraints minskar risken för ogiltiga värden. En utmaning var att tänka igenom vilka relationer som ska vara 1-n och var en constraint som UNIQUE eller CHECK faktiskt tillför kvalitet. JOIN-frågorna gjorde det tydligt hur information från flera tabeller kan kombineras utan att duplicera samma information i varje rad. LINQ var också en bra jämförelse eftersom samma logik kan uttryckas på ett mer objektorienterat sätt i C#. Ett möjligt förbättringsområde är att lägga till fler tabeller, exempelvis Playlist och User, samt fler avancerade queries och index när datamängden växer. I ett riktigt projekt skulle även migrationshantering, testdata, backupstrategi och mer detaljerad behörighetsstyrning behöva utvecklas vidare.

13. Slutsats

Projektet uppfyller kraven för CRUD, relationsdatabas, JOINS, modellering, SQL och LINQ. Strukturen är medvetet enkel men använder constraints och relationer för att hålla datan konsekvent. Lösningen kan byggas vidare med fler entiteter och mer avancerad funktionalitet i ett större backendprojekt.